From Entropy to Epiplexity

Rethinking Information for Computationally Bounded Intelligence

Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J. Zico Kolter, Andrew Gordon Wilson

Carnegie Mellon University, New York University

arXiv:2601.03220, version 2, 16 March 2026

Speaker: Alexey Kravatskiy

Three apparent paradoxes of information

1. Deterministic computation cannot increase information

Yet self-play and synthetic data can improve a model's capabilities.

2. Total information does not depend on prediction order

Yet language-model loss can differ between forward and reverse text.

3. Likelihood modelling targets the data distribution

Yet prediction can require inference absent from the generating procedure.

Section 5, Finzi et al. (2026). These are apparent tensions, not contradictions of the classical results.

Shannon entropy and Kolmogorov complexity

Shannon entropy: a distribution

$$p(x) = \Pr(X = x),$$
$$H(X) = \mathbb{E}[-\log_2 p(X)].$$

For discrete X, entropy is a lower bound on the expected length of any binary prefix code.

Uniform n-bit strings: $H(X) = n$.

A fixed string: $H(X) = 0$.

Kolmogorov complexity: a string

$$K(x) = \min_{p:\, \mathcal{U}(p)=x} |p|.$$

The shortest self-delimiting program that outputs x and halts on a fixed prefix-free universal Turing machine $\mathcal{U}$.

An alternating string is specified by its rule and length. Most uniform n-bit strings require at least $n - O(1)$ program bits.

$K(x)$ is uncomputable in general. Definition 1 and Section 2.1, Finzi et al. (2026).

Paradox 1: deterministic computation and new information

For discrete X and a fixed computable transformation f,

$$H(f(X)) \le H(X), \qquad K(f(x)) \le K(x) + K(f) + c.$$

The output has a description: specify the input and the program for f, then run it. These bounds do not restrict how long that computation takes.

Example: expanding a random seed

A pseudorandom generator transforms k seed bits into $n > k$ output bits. The output still has entropy at most k and a description of length at most $k + K(G) + O(\log_2 k)$. For a secure generator, no randomized test with runtime polynomial in k distinguishes the output from uniform data with non-negligible advantage.

AlphaZero learns a policy and a value function through self-play. Specifying its algorithm and random seed can take far fewer bits than storing the trained network, but reconstructing that network requires running the training procedure.

Negligible advantage: eventually smaller than k^{-m} for every fixed $m > 0$. Section 5.1 and Theorem 12, Finzi et al. (2026); the theorem uses polynomial-length advice independent of the sampled seed.

Paradox 2: equal information, different prediction costs

The chain rule gives the same joint entropy in either order:

$$H(X) + H(Y \mid X) = H(X, Y) = H(Y) + H(X \mid Y).$$

Each order describes the same pair. The identity does not bound the cost of evaluating or sampling from the conditional distributions.

Example: a bijection computable in polynomial time

Let X be uniform on n-bit strings and $Y = f(X)$. Since f is a bijection, Y is also uniform and both conditional entropies are zero.

- **Input first:** encode X in n bits, then compute $Y = f(X)$.
- **Output first:** encode Y in n bits, then recover $X = f^{-1}(Y)$.

One-wayness means that no randomized algorithm with runtime polynomial in n recovers X from $f(X)$ with non-negligible probability over uniform X.

Prediction order can therefore affect what a model can compute. The paper studies this with cellular-automaton pairs and reordered chess data.

Section 5.2 and Theorem 13, Finzi et al. (2026). The theorem also allows polynomial-length advice independent of the sampled input; one-wayness is an assumption.

Paradox 3: can prediction require more than generation?

Likelihood training minimizes

$$L_Q(P) = \mathbb{E}_{Y \sim Q}[-\log_2 P(Y)].$$

When the model class contains the data distribution Q, the minimum is attained at $P = Q$.

Does matching the distribution require only reproducing the generator's computations? Consider a mystery story:

The author generates the story

Choose the culprit, then write consistent clues. The culprit is already known.

The model predicts the reveal

Given only the clues, assign probabilities to possible culprits before the reveal.

Prediction requires inference from clues to culprit; generation does not. The paper calls this **induction**: inference about unobserved variables. Under a time bound, it may require a different algorithm from the generator's.

Section 5.3 and the mystery-story example in Section 5.3.1, Finzi et al. (2026).

Time-bounded probabilistic models

Fix a prefix-free universal Turing machine. A program P implements the same distribution P in two modes: evaluate $P(x)$ or sample $x \sim P$. Each call must finish within $T(n)$ steps, for every input.

Example: one autoregressive model, fixed parameters θ

The network outputs conditional probabilities $p_\theta(x_j \mid x_{<j})$.

- **Evaluation:** supply the observed prefixes and multiply:

$$P_\theta(x) = \prod_{j=1}^{n} p_\theta(x_j \mid x_{<j}).$$

- **Sampling:** draw x_1, then $x_2 \mid x_1$, and so on, using the same conditionals and fresh random bits.

$\mathcal{P}_T$ is the set of programs satisfying both runtime bounds and whose sampler agrees with the evaluated probabilities. $|\mathrm{P}|$ is the program length in bits.

Definition 7, Finzi et al. (2026): T is non-decreasing and time-constructible; probabilities are exact, normalized and have finite binary expansions.

Epiplexity: the two-part description

For X on $\{0,1\}^n$, choose an admissible model minimizing the expected two-part description length:

$$\mathrm{P}^\star \in \arg\min_{\mathrm{P}\in\mathcal{P}_T} \big\{ |\mathrm{P}| + \mathbb{E}[-\log_2 P(X)] \big\}.$$

Among minimizers, select one with the shortest program.

Epiplexity: model description

$$S_T(X) := |\mathrm{P}^\star|.$$

Length of the selected model's program.

Time-bounded entropy: data description

$$H_T(X) := \mathbb{E}[-\log_2 P^\star(X)].$$

Expected data-code length using that model.

$$\mathrm{MDL}_T(X) = S_T(X) + H_T(X).$$

Definition 8, Finzi et al. (2026).

Examples: random bits and a repeated pattern

Assume the length n is specified and the time bound permits the models below.

Uniform bits: $X \sim U_n$

A constant-length program assigns $P(x) = 2^{-n}$ to every n-bit string.

$$\mathrm{MDL}_T(X) \le n + c, \qquad H_T(X) \ge H(X) = n.$$

Consequently $S_T(X) \le c$: a large data-code length can accompany a small model description.

Alternating strings: two equally likely outcomes

$X = 0101\ldots$ or $1010\ldots$, each with probability $\frac{1}{2}$. A constant-length program generates either pattern in linear time.

$$S_T(X) = O(1), \qquad H_T(X) = O(1).$$

Cryptographic consequences of the time bound

- **Pseudorandom generator.** G maps k random seed bits to $n > k$ output bits. $H(G(U_k)) \leq k$, yet for a secure generator $H_{\mathrm{Poly}}(G(U_k)) \approx n$ and $S_{\mathrm{Poly}}(G(U_k)) = O(1)$.

- **One-way function.** Computing $f(x)$ takes polynomial time. No randomized polynomial-time algorithm finds a preimage with non-negligible probability over uniformly sampled inputs. For a one-way *permutation*, the paper proves that the two factorization orders have different time-bounded entropies.

Theorems 9 and 13, Finzi et al. (2026). These results assume security against non-uniform probabilistic polynomial-time algorithms.

Conditional epiplexity: predicting labels from inputs

For supervised data (X, Y), the decoder already has the input X. The model evaluates $P(y \mid x)$ and samples $Y \mid X = x$ within the bound for each fixed x.

$$\mathrm{P}^{\star}_{Y|X} \in \arg\min_{\mathrm{P} \in \mathcal{P}^{X}_{T}} \{|\mathrm{P}| + \mathbb{E}_{X,Y}[-\log_2 P(Y \mid X)]\}\,.$$

$$S_T(Y \mid X) := |\mathrm{P}^{\star}_{Y|X}|, \qquad H_T(Y \mid X) := \mathbb{E}_{X,Y}[-\log_2 P^{\star}(Y \mid X)].$$

Example: image classification

The model code describes the classifier. The data code describes the labels given the images; it does not encode the images.

In general, $H_T(Y \mid X) \neq H_T(X, Y) - H_T(X)$.

Definition 11, Finzi et al. (2026).

Compute budget for training and evaluation

The experiments restrict the search to models trained with a chosen architecture and optimiser. To decode the empirical two-part code, first reconstruct the weights by replaying training, then evaluate the target data with those weights. Let N count parameters, D training tokens and $\mathcal{D}$ evaluation tokens.

$$\underbrace{6ND}_{\text{replay training: } 2ND+4ND} + \underbrace{2N\mathcal{D}}_{\text{evaluation: forward only}} \leq T.$$

- A dense forward pass costs roughly $2N$ FLOPs per token: one multiplication and one addition per parameter.
- Backpropagation computes gradients for activations and weights: roughly $4N$ FLOPs per token, twice the forward pass.

The terms add because the decoder performs these stages in sequence. Sampling does not involve training; after the weights are reconstructed, it is a separate call that must satisfy its own sampling bound. For each (N, D) satisfying the budget, estimate the model-code length and add the validation loss (bits per token) multiplied by $\mathcal{D}$. Retain the run with the smallest sum.

Sections 3–4 and Appendix B.1, Finzi et al. (2026). This is empirical compute accounting. Formal membership in $\mathcal{P}_T$ requires both evaluation and sampling to satisfy the bound separately.

Prequential coding: transmit data and reproduce training

Both sides share the initial training state θ_0 (weights and optimiser state), update rule A, schedule and random seed. Let Z_i be the next training batch and $P_i = P_{\theta_i}$ the model before its update.

1. **Encoder:** arithmetic-code Z_i under P_i; transmit the resulting bits.
2. **Decoder:** use its identical P_i and those bits to recover exactly Z_i.
3. **Both:** compute $\theta_{i+1} = A(\theta_i, Z_i)$, then repeat.

$$L_{\text{preq}} = \sum_{i=0}^{M-1} -\log_2 P_i(Z_i).$$

The message reconstructs the exact training batches *and* the final state θ_M.

Ideal lengths omit arithmetic-coding and setup overhead. Prequential approach: Dawid (1984); neural-network coding: Blier and Ollivier (2018).

Estimating model bits from the training-loss curve

Let $Z = (Z_0, \ldots, Z_{M-1})$. The prequential message reconstructs (Z, P_M); given P_M, the data alone have a code of length L_{final}.

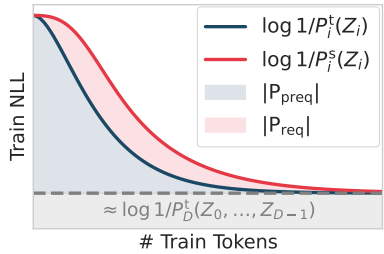
The unbounded algorithmic chain rule suggests

$$K(Z, P_M) \approx K(P_M) + K(Z \mid P_M).$$

Substituting the two code lengths gives

$$L_{\text{final}} = \sum_{i=0}^{M-1} -\log_2 P_M(Z_i),$$

$$\widehat{K}_{\text{preq}} = L_{\text{preq}} - L_{\text{final}}.$$



This is a **heuristic**: subtracting two upper bounds need not bound $K(P_M)$, and the chain rule gives no decoder within T.

Section 4.1 and Fig. 2a, Finzi et al. (2026). The unbounded algorithmic chain rule has logarithmic overhead in this form; it does not give a time-bounded identity.

Requential coding: reconstruct a model from sample indices

At step i, the encoder has a teacher checkpoint Q_i, typically trained on real data. Both sides share a student $P_i = P_{\theta_i}$, initial state, update rule A and random seed s.

1. Generate indexed proposals $Y_i^{(j)} \sim P_i$, reproducible from (s, i, j).
2. The encoder uses **relative entropy coding** to select J_i so that $\widetilde{Z}_i = Y_i^{(J_i)} \sim Q_i$, then sends a prefix code for J_i.
3. The decoder regenerates $\widetilde{Z}_i$. Both update $\theta_{i+1} = A(\theta_i, \widetilde{Z}_i)$; repeating reconstructs the final student.

If $Q_i = P_i$, the first proposal suffices; its random bits need no transmission.

$$d_i := \mathrm{KL}(Q_i \| P_i) = \mathbb{E}_{Z \sim Q_i}\left[\log_2 \frac{Q_i(Z)}{P_i(Z)}\right],$$

$$\mathbb{E}[\ell_i \mid \text{history}] \le d_i + 2\log_2(1 + d_i) + \kappa, \qquad \kappa < 5.21.$$

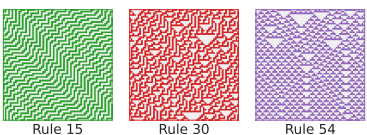
Here ℓ_i is the index-message length. The leading model-length estimate is $\widehat{K}_{\mathrm{req}} = \sum_i d_i$.

Relative entropy coding: Theis and Ahmed (ICML, 2022). Protocol and bound: Qiu et al. (2026), Section 3.1 and Appendix A.1. Assume $P_i(z) > 0$ whenever $Q_i(z) > 0$; setup and logarithmic overhead are omitted from the leading estimate.

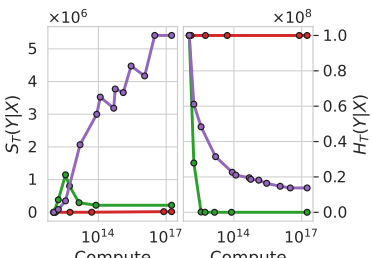
Cellular automata: simple dynamics, different model parts

An *elementary cellular automaton* (ECA) updates binary cells in parallel from their three-cell neighbourhoods (Wolfram, 2002).

Setting. 64 periodic cells; $Y = F_r^{48}(X)$; transformer sweep; requential code.



Rule	15	30	54
$S_T(Y \mid X)$, bits	2×10^5	≈ 0	5.4×10^6
$H_T(Y \mid X)$, bits	≈ 0	10^8	1.5×10^7

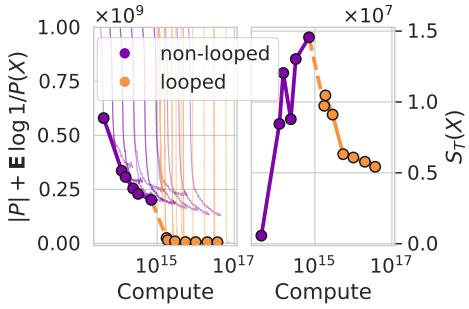


Left: rollouts, time downward. Right: S_T and H_T against compute in FLOPs. Table: our estimates from the plots at 10^{17} FLOPs. Source: Fig. 3 of Finzi et al. (2026).

Epiplexity need not increase with compute

Setting. ECA rule 54, 64 updates. The non-looped model predicts the final state; the looped model also predicts intermediate states.

- Below 10^{15} FLOPs, the non-looped model gives the shorter estimated code; its model part reaches 1.45×10^7 bits.
- Above that budget, the looped model gives the shorter code. Repeated application of the simple dynamics becomes preferable.
- The authors interpret this as emergence: limited compute favours prediction from large-scale patterns.

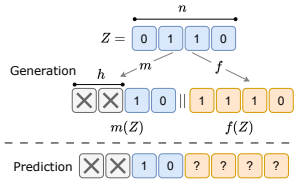


Two-part code and S_T against compute in FLOPs. Source: Fig. 6 of Finzi et al. (2026).

Paradox 3 in the experiment: hidden inputs

Generate data: start with a uniformly random row Z of 32 binary cells on a ring.

- m_h deletes the first h cells ($h = 0, \ldots, 5$); the observed input is $X = m_h(Z)$.
- f applies rule 30 to the *complete* row four times; the target is $Y = f(Z)$.



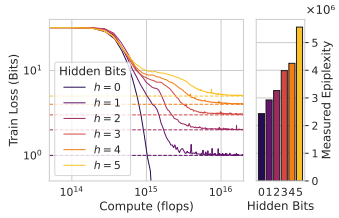


Fig. 5b, Finzi et al. (2026): prediction loss against compute (dashed: reported h-bit baseline), and estimated S_T.

- A transformer predicts Y given X; requential coding estimates its model part. With h hidden cells, 2^h complete initial rows are consistent with X.
- More hidden cells require more training compute to reach the baseline and increase the model-part estimate: evidence of inference absent from the generator.

Section 5.3.1. The plots do not identify the learned algorithm or establish a complexity lower bound.

Experiment: the order of chess data

Setting. Lichess games in two orders: moves → final board, or board → moves. Transformer sweep with requential coding.

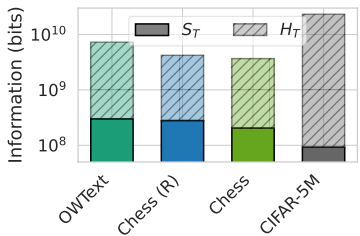


Left: H_T and S_T against compute in FLOPs. Right: S_T and accuracy after fine-tuning. Source: Figs. 4c and 7 of Finzi et al. (2026).

- At large compute, reverse order gives higher H_T and S_T.
- After fine-tuning, reverse-order models reach 0.28 board-evaluation accuracy versus 0.21 for forward-order models; puzzle accuracy is similar.

Application: structure in natural data

Setting. Equal token budgets and a shared compute bound. Compare OpenWebText, chess and CIFAR-5M with the requential code.



Chess (R): reverse order. Source: Fig. 8a of Finzi et al. (2026). Table: our estimates from the figure.

Dataset	S_T, bits	MDL_T, bits	S_T/MDL_T
OpenWebText	3×10^8	7×10^9	4%
Chess, reverse	2.8×10^8	4×10^9	7%
Chess, forward	2×10^8	3.5×10^9	6%
CIFAR-5M	9×10^7	2.3×10^{10}	0.4%

- The datasets use the same evaluation-token count and compute bound. Images are encoded as pixel tokens; chess and text use their respective tokenisations.
- The estimates depend on the tokenisation, model family and compute budget.

Conclusions and limitations

Summary

- S_T counts selected model bits; H_T is the expected data-code length.
- Computational restrictions can change both the total description length and its decomposition.
- Epiplexity can be unbounded: some families satisfy $S_{\mathrm{Poly}}(X_n) = \Omega(\log_2 n)$ under one-way-function assumptions.
- Neural-network estimates combine a model-code estimate with validation loss, optimized under a compute budget.

Limitations

- S_T is not a guarantee of downstream usefulness.
- Estimates depend on architecture, optimiser, T and dataset size.

Follow-up work from Wilson's group

Qiu et al. (2026): **Requential Coding**

A teacher selects examples drawn from the student's predictive distribution. Recording the selection indices reconstructs the student's training trajectory; the coding cost depends on teacher–student disagreement. This yields PAC–Bayes generalization bounds for billion-parameter language models. At fixed loss, larger models can have shorter codes.

Su et al. (2026): **EpiSelect and EpiGen**

EpiSelect fits cross-domain loss-scaling laws and adjusts sampling weights to maximize predicted gains in an online epiplexity proxy. **EpiGen** trains a synthetic-data generator with a policy-gradient reward: the reduction in learner loss on earlier samples after training on a new batch. Both methods report improved downstream performance.

Literature: epiplexity and follow-up work

Marc Finzi, Shikai Qiu, Yiding Jiang, Pavel Izmailov, J. Zico Kolter, Andrew Gordon Wilson (2026).
From Entropy to Epiplexity.

Shikai Qiu, Marc Finzi, Yujia Zheng, Kun Zhang, Andrew Gordon Wilson (2026).
Requential Coding.

Ellen Su, Andres Potapczynski, Shikai Qiu, Edward Hughes, Andrew Gordon Wilson (2026).
Epiplexity Guided Data Selection and Generation for Out-of-Distribution Generalization.

Literature: background

Ming Li and Paul Vitányi (2008).
An Introduction to Kolmogorov Complexity and Its Applications.

Stephen Wolfram (2002).
A New Kind of Science.

Yiding Jiang, Allan Zhou, Zhili Feng, Sadhika Malladi, J. Zico Kolter (ICLR, 2025).
Adaptive Data Optimization: Dynamic Sample Selection with Scaling Laws.

Literature: sequential and relative entropy coding

A. Philip Dawid (1984).
Present Position and Potential Developments: Some Personal Views. Statistical Theory: The Prequential Approach.

Léonard Blier and Yann Ollivier (NeurIPS, 2018).
The Description Length of Deep Learning Models.

Lucas Theis and Noureldin Y. Ahmed (ICML, 2022).
Algorithms for the Communication of Samples.

Questions for the audience: answers

1. Let $X \sim U_n$. Which quantity is close to n: $S_T(X)$ or $H_T(X)$?

Answer

$H_T(X) = n + O(1)$. A constant-length uniform model gives $\mathrm{MDL}_T(X) \leq n + c$; since $H_T(X) \geq n$, the selected model has $S_T(X) \leq c$.

2. In the prequential estimate, which part of the training-loss curve approximates the epiplexity?

Answer

The cumulative excess loss above the final-model baseline: $\sum_i [-\log_2 P_{\theta_i}(Z_i) + \log_2 P_{\theta_M}(Z_i)]$.

3. If $P = Q$, must the model use the same algorithm as the generator?

Answer

No. The author chooses a culprit, then writes the clues. The predictor receives the clues and infers the culprit. The same story distribution can require different computations for generation and prediction.

Backup: selecting a proposal by rejection sampling

At one step, let P be the student and Q the teacher. Choose $B \geq \max_z Q(z)/P(z)$, with $P(z) > 0$ whenever $Q(z) > 0$.

1. The shared seed generates indexed proposals $Y^{(j)} \sim P$ and independent $U_j \sim \mathrm{Uniform}[0, 1]$.
2. The encoder accepts the first index J satisfying

$$U_J \leq \frac{Q(Y^{(J)})}{B\,P(Y^{(J)})}.$$

3. Transmit J; the decoder regenerates $Y^{(J)}$ without knowing Q.

For a proposal, $\Pr(Y = z,\ \text{accept}) = P(z)\,Q(z)/(BP(z)) = Q(z)/B$. Conditioning on acceptance therefore gives distribution Q.

More sophisticated REC constructions provide the KL-based length bound on the main slide.

Qiu et al. (2026), Fig. 2 and Section 2; Theis and Ahmed (2022). Simple rejection sampling can have a much larger coding cost.

Backup: basic properties

Proposition 1 (Sec. 3 of the paper)

Let c_1, c_2 be constants independent of X. For a random variable X on $\{0,1\}^n$:

1. $S_T(X) \geq 0$ and $H_T(X) \geq 0$;
2. $H(X) \leq \mathrm{MDL}_T(X) \leq n + c_1$;
3. $\mathrm{MDL}_{T'}(X) \leq \mathrm{MDL}_T(X)$ whenever $T'(n) \geq T(n)$;
4. $\mathrm{MDL}_{T'}(f^{-1}(X)) \leq \mathrm{MDL}_T(X) + |\mathrm{f}| + c_2$ with $T'(n) = T(n) + \mathrm{Time(f)}$, for a program f that computes a bijection f in fixed time.

Property 4 is the analogue of $K(f(x)) \leq K(x) + K(f) + c$. Under a time bound a short program for f^{-1} gives no short program for f.

Theorem 1 (Thm. 10 of the paper)

If one-way functions secure against probabilistic polynomial-time algorithms with polynomial advice exist, then some random variables X_n on $\{0,1\}^n$ have $S_{\mathrm{Poly}}(X_n) = \Omega(\log_2 n)$.